AI HOME MANAGER

Project Proposal

Team: Vibe Crew

Team Members: Ananya Chakraborty • Dheeraj Kumar Vishwakarma • Narayan Lal Suthar • Pratik Dey • Sanyam Mittal

Problem Statement

Households rely on disconnected apps, chats, spreadsheets, and reminders to manage chores, groceries, bills, maintenance, and calendars, resulting in fragmented coordination and increased mental load. Existing solutions lack unified cross-domain reasoning and fail to address different members' roles, responsibilities, and privacy needs. The proposed **AI Home Manager** is a web-based, text-driven application that uses specialized AI agents under a central orchestrator to manage household activities, maintain shared household intelligence, and enforce strict **role-based, per-member access control at the data layer**.

Objective

The objective of this project is to build a multi-agent AI Home Manager in which specialized agents for tasks, inventory, maintenance, calendar, and expenses collaborate under a central orchestrator using shared household knowledge, while enforcing strict, role-based, per-member access control at the data layer.

Data Description

Household members interact with the system through natural-language chat and unstructured document uploads such as receipts, bills, and other household documents. Chat inputs are processed as raw text, while uploaded PDFs and images are parsed into structured fields. All data is normalized into a single SQLite3 relational schema spanning six domains: **People & Permissions**, **Tasks & Schedule**, **Food & Inventory**, **Finance & Budgeting**, **Home & Maintenance**, and **Documents**.

Data processing follows a deterministic seven-step pipeline orchestrated by LangGraph and served through Flask: **Capture** receives the message or document, **Understand** extracts relevant intents and entities, **Authorize** performs role-based access checks at the database layer before LLM access, **Update** writes validated data to SQLite3, **Plan** performs cross-domain reasoning, **Act** executes the required tasks, and **Remember** persists the outcome to long-term state.

System Architecture

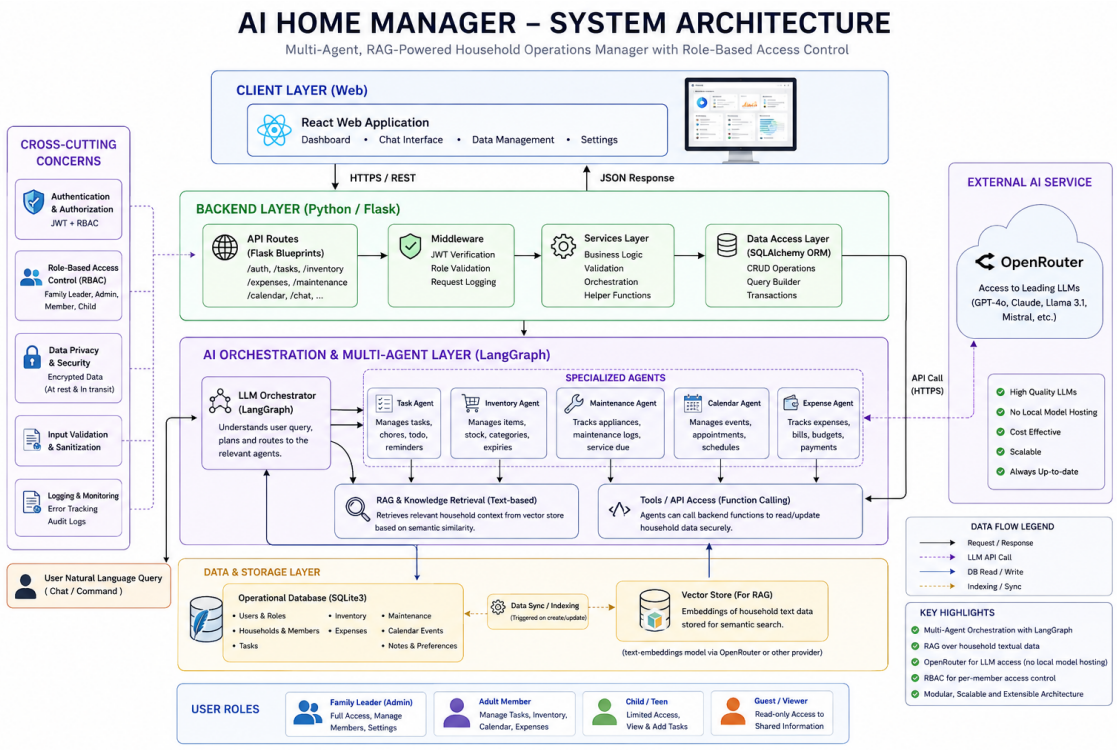


Figure 1: High-Level System Architecture

Tech Stack

Table 1: Technology Stack

Layer	Technology	Purpose
Frontend	React.js	Web interface, dashboard, and conversational chat
Backend	Python + Flask	REST APIs, business logic, authentication, and data access
Database / ORM	SQLite3 + SQLAlchemy	Store and manage household data
LLM Access	OpenRouter API	Natural-language understanding and generation
Agent Orchestration	LangGraph	Build and coordinate the multi-agent workflow
RAG	Embeddings + Vector Store	Retrieve relevant household context for LLM responses
Authentication & Authorization	JWT + RBAC	Secure authentication and per-member access control
API Communication	REST / JSON	Communication between React frontend and Flask backend
Development	Git	Version control

Key AI/LLM Capabilities

1. **Multi-Agent Collaboration:** Specialized agents for tasks, inventory, maintenance, calendar, and expenses collaborate through LangGraph to handle different household needs.
2. **Natural Language Understanding:** The system understands natural-language requests and extracts relevant intents, entities, dates, people, and actions.
3. **Cross-Domain Reasoning:** The system combines information from multiple household domains to answer complex queries rather than treating each activity independently.
4. **Retrieval-Augmented Generation (RAG):** Relevant information is retrieved from household documents and stored knowledge to ground LLM responses in the user's actual household data.
5. **Context-Aware Personalization:** Responses are tailored according to the user's household role, permissions, existing tasks, schedules, and available information.
6. **Task Planning and Execution:** The AI breaks complex requests into multiple steps, delegates them to appropriate agents, and executes the resulting actions.
7. **Role-Aware Responses:** The system considers member permissions when retrieving information and generating responses, ensuring that users only receive information they are authorized to access.

What Makes It Innovative?

While existing family-management applications increasingly combine features such as calendars, chores, groceries, and budgeting, the proposed system focuses on how these domains can be intelligently connected. Its key innovation is a **multi-agent architecture** where specialized agents collaborate through a central orchestrator and reason over a shared household state. The system further combines **cross-domain reasoning, RAG-based household knowledge, and fine-grained per-member access control**, allowing different family members to receive personalized responses without exposing information outside their permitted access.

Conclusion

The proposed AI Home Manager aims to transform fragmented household management into a unified, intelligent, and secure experience. By coordinating specialized AI agents and maintaining shared household context while respecting member-specific permissions, the system can provide personalized assistance across everyday household activities. The web-based, text-driven application demonstrates a practical use of LLMs for intelligent household coordination.

Basic UI Design: The initial interface design is available in the Figma Design.